

WarehouseFlow Video Script

This script is written for an individual screen-recording video where you narrate the project and demonstrate the working system. It is designed for about 6 to 8 minutes.

Before Recording

Have these ready:

- Local stack running:
 - `docker compose up -d`
 - routing service on `http://localhost:8082`
 - ingestion service on `http://localhost:8081`
- Browser tabs ready:
 - [README.md](#)
 - [dashboard/index.html](#)
 - [results/charts/exp1_ecs_scaling.png](#)
 - [results/charts/exp2_resilience.png](#)
 - [results/charts/exp3_contention.png](#)
 - [results/charts/exp5_kafka_partitions.png](#)
 - `http://localhost:8081/health`
 - `http://localhost:8082/health`
 - `http://localhost:3000`
 - terminal with the local logs or `docker ps`

Optional terminal commands to keep ready:

```
Invoke-WebRequest -UseBasicParsing http://localhost:8081/health | Select-Object -
ExpandProperty Content
Invoke-WebRequest -UseBasicParsing http://localhost:8082/health | Select-Object -
ExpandProperty Content

$body='{ "customer_id": "video-demo", "sku": "SKU-ALPHA", "quantity": 1, "region": "US-
EAST" }'
Invoke-WebRequest -UseBasicParsing -Uri http://localhost:8081/api/v1/orders -
Method POST -ContentType 'application/json' -Body $body | Select-Object -
ExpandProperty Content

docker exec warehouseflow-postgres psql -U warehouseflow -d warehouseflow -c
"SELECT order_id, customer_id, sku, warehouse_id, picker_id, status, latency_ms
FROM routing_decisions ORDER BY id DESC LIMIT 5;"
```

Recording Script

Segment 1: Intro and Problem Framing

Show:

- Start on the repo README near the title and architecture summary

Say:

"Hi, this is my final project, WarehouseFlow. It is a distributed real-time order routing engine inspired by warehouse management systems. The system takes incoming orders, publishes them to Kafka, routes them to one of three warehouse nodes, updates inventory atomically in Redis, and stores every routing decision in Postgres for correctness and analysis."

"The goal of this project was not just to build a working microservice pipeline, but to study distributed systems tradeoffs such as horizontal scaling, failure handling, contention, Kafka partitioning, noisy-neighbor effects, and cold starts."

Segment 2: High-Level Architecture

Show:

- Stay on README architecture section
- Slowly scroll through project structure

Say:

"At a high level, the ingestion service exposes an HTTP API. It validates incoming orders and pushes them to Kafka. The routing service consumes those Kafka messages, checks warehouse availability, inventory, and picker capacity, then makes a routing decision and writes it to Postgres."

"Each warehouse has its own Redis instance, which makes it possible to simulate partial failures and analyze resilience behavior independently. I also built a local Docker Compose environment, Terraform for AWS, Prometheus and Grafana monitoring, and a standalone dashboard for demo purposes."

Segment 3: Show the System Is Running

Show:

- Terminal with `docker ps`
- Browser tabs for `/health`
- If possible, briefly show Grafana or Prometheus tab open

Say:

"Here I'm showing the local stack running. Redpanda is my Kafka-compatible broker, I have three Redis nodes, Postgres, Prometheus, and Grafana. On top of that, the two Go services are running locally."

"These two health endpoints confirm that both the ingestion service and the routing service are up and healthy."

Segment 4: End-to-End Demo

Show:

- Terminal
- Run the order submission command

- Then run the Postgres query showing the audit row

Say:

"Now I'll submit one order through the ingestion API. The ingestion service accepts the request, assigns an order ID, and returns a queued response."

"After the message is published to Kafka, the routing service consumes it, selects a warehouse based on region-aware preference and resource availability, and writes the final routing decision into Postgres."

"Here in the audit log you can see the order ID, the selected warehouse, the picker that was assigned, the routing status, and the measured latency."

Segment 5: Key Implementation Decisions

Show:

- Open `routing-service/store/redis.go`
- Briefly highlight the Redis Lua decrement logic
- Open `routing-service/router/engine.go`

Say:

"One of the most important implementation decisions was using a Lua script in Redis for inventory decrement. That means inventory checks and decrements happen atomically inside Redis, which prevents oversells even under concurrency."

"The base routing engine prioritizes warehouses by region, pings the warehouse, checks inventory, claims a picker, decrements inventory, and then asynchronously releases the picker after simulated work."

"I also implemented optimistic and pessimistic concurrency strategies to compare performance under high contention."

Segment 6: Resilience Patterns

Show:

- Open `routing-service/resilience/`
- Briefly mention circuit breaker, bulkhead, retry, fail-fast, degradation
- Optionally show the dashboard next

Say:

"To make the routing service more production-realistic, I added a resilience package with five patterns: circuit breaker, bulkhead, retry with jitter, fail-fast timeouts, and graceful degradation."

"These components are individually testable and were also tied back to the experiments. For example, the circuit breaker is especially important when a warehouse becomes slow instead of fully crashed, because it prevents one bad dependency from blowing up overall tail latency."

Segment 7: Dashboard Demo

Show:

- Open `dashboard/index.html`
- Click `START LOAD`
- Optionally click `CRASH WAREHOUSE B` and `RECOVER ALL`

Say:

"This dashboard is a self-contained simulator I built for presentation. It gives a visual explanation of the system with routing particles, warehouse states, key metrics, and experiment controls."

"When I start the simulated load, you can see traffic flow through the system. If I crash Warehouse B, the dashboard shows the system degrading and then recovering. This made it much easier to explain distributed-system behavior during failures without depending on a live cloud deployment."

Segment 8: Experiments Overview

Show:

- Open `exp1_ecs_scaling.png`

Say:

"For the experiments, I focused on concrete tradeoffs instead of generic benchmarks. This first chart is the horizontal scaling experiment. The main result was that scaling from one to four routing tasks improved throughput and reduced tail latency significantly, but after that the bottleneck shifted to shared cache resources."

Show:

- Open `exp2_resilience.png`

Say:

"This experiment shows resilience under warehouse failure. When one warehouse failed, the system degraded briefly, failover happened in a few seconds, and no orders were permanently lost. The dead-letter queue absorbed in-flight failures and those orders could be recovered after restoration."

Show:

- Open `exp3_contention.png`

Say:

"This was one of the most interesting experiments. I compared optimistic versus pessimistic concurrency under scarce inventory. The surprising result was that both achieved zero oversells. The real correctness guarantee came from the atomic Lua decrement in Redis. The strategies mainly changed throughput and latency behavior, not correctness itself."

Show:

- Open `exp5_kafka_partitions.png`

Say:

"This chart highlights another important distributed systems lesson: consumer scaling is bounded by Kafka partition count. Adding more consumers than partitions gives idle workers and no additional benefit."

Segment 9: AWS Deployment Status

Show:

- Open `terraform/main.tf`
- Optionally show terminal snippet or mention

Say:

"I also prepared full Terraform infrastructure for AWS including VPC, ECS, ALB, RDS, ElastiCache, SQS, and ECR. I validated and corrected the Terraform so it plans cleanly in the learner-lab environment and reuses the existing LabRole for ECS."

"I attempted a full apply, and the deployment progressed through networking, load balancing, databases, Redis, ECR, and ECS cluster creation, but it failed at Amazon MSK because the learner-lab account does not allow `kafka:CreateCluster`. I then destroyed the partial infrastructure to avoid leaving resources running."

"So the cloud deployment path is operational up to the account-permission boundary, and the local end-to-end system is fully working."

Segment 10: Closing Reflection

Show:

- Return to README or final report

Say:

"The main thing I learned from this project is that distributed systems tradeoffs become much clearer when you force yourself to build experiments around them. Horizontal scaling, tail latency, contention, failure recovery, and partitioning are much easier to understand when you can actually observe them in a working system."

"If I continued this project, my next steps would be replacing the MSK dependency with a deployment option allowed in the lab account, improving health-check speed, and expanding the cloud path to support the same resilience experiments directly in AWS."

"That's WarehouseFlow. Thank you."